

Procedural Programming using C -23CSE201

Lab Manual

Submitted By

NAME	P. BHANU KIRAN
ROLL NO	AV.SC.U4CSE25142
YEAR/SEM/SECTION	2nd YEAR/ SEM-3/CSE-B
CREDITS	04

Submitted to

NAME	K. VENKATA ANUSHA
DESIGNATION	ASST.PROFFESSOR
DEPARTMENT	SCHOOL OF COMPUTING

Index

Sno	Name	Level	Submitted On	Page no
1	Max Consecutive Ones	Easy	Sep 29, 2026 22:59	4
2	Contains Duplicate	Easy	Sep 28, 2026 21:59	6
3	Concatenation of Array	Easy	Sep 27, 2026 23:38	8
4	Flipping an Image	Easy	Oct 04, 2026 22:59	10
5	Maximum Number of Words Found in Sentences	Easy	Oct 04, 2026 23:11	12
6	Rotate Array	Medium	Sep 30, 2026 21:42	14
7	Reverse Words in a String	Medium	Oct 02, 2026 23:24	16
8	Container With Most Water	Medium	Oct 03, 2026 23:08	18
9	Rotate Image	Medium	Oct 04, 2026 22:15	20
10	Longest Substring Without Repeating Characters	Medium	Oct 05, 2026 22:53	22

This Page is intentionally left blank.

Description – Max Consecutive Ones

Given a binary array nums, return *the maximum number of consecutive* 1*'s in the array*.

Example 1:

Input: nums = [1,1,0,1,1,1]

Output: 3

Explanation: The first two digits or the last three digits are consecutive 1s. The maximum number of consecutive 1s is 3.

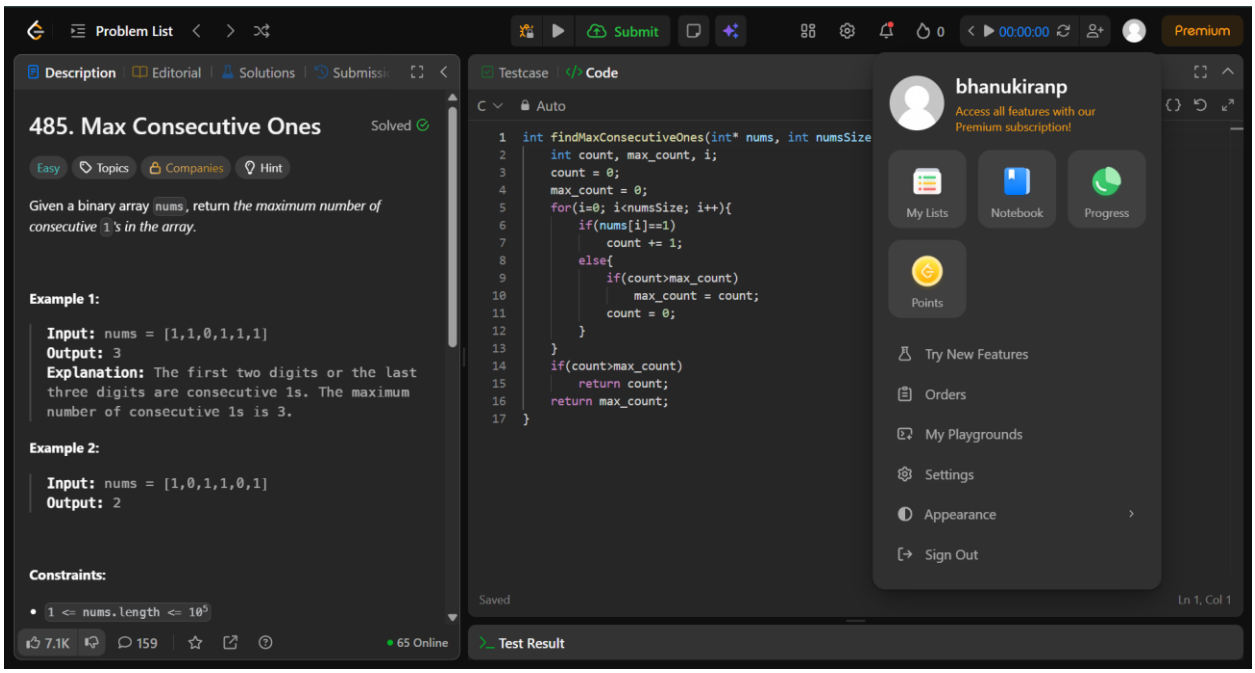
Example 2:

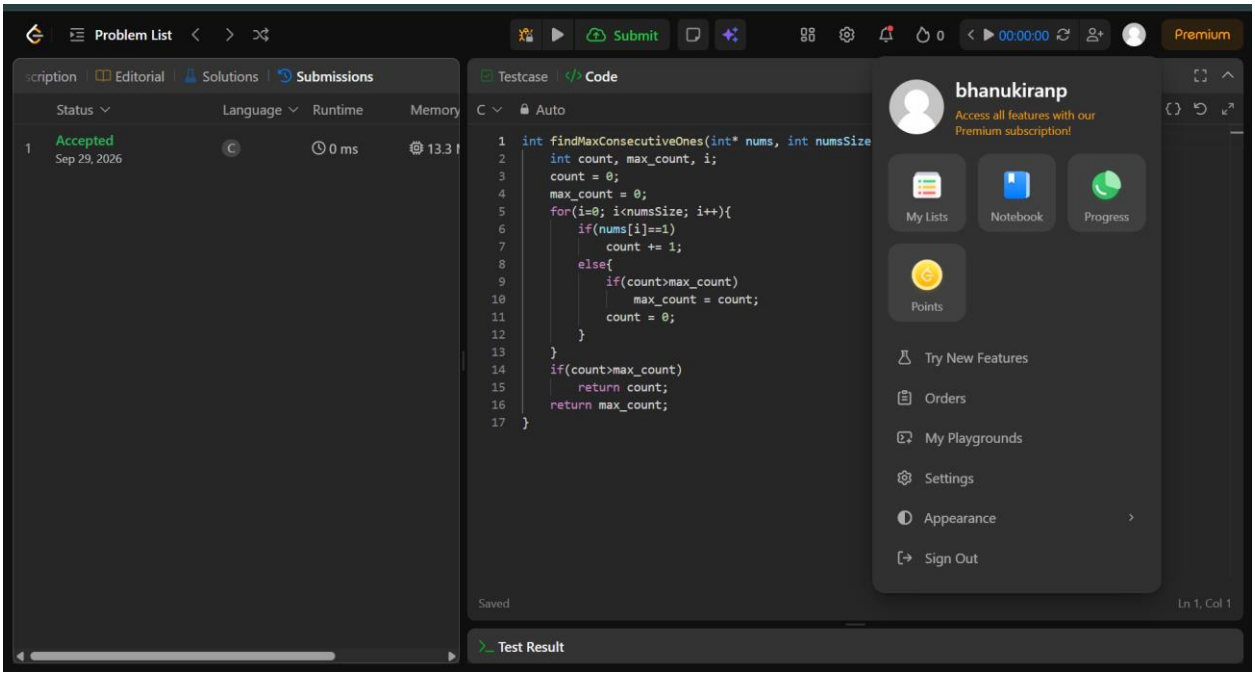
Input: nums = [1,0,1,1,0,1]

Output: 2

Code:

```
int findMaxConsecutiveOnes(int* nums, int numsSize) {
    int count, max_count, i;
    count = 0;
    max_count = 0;
    for(i=0; i<numsSize; i++){
        if(nums[i]==1)
            count += 1;
        else{
            if(count>max_count)
                max_count = count;
            count = 0;
        }
    }
    if(count>max_count)
        return count;
    return max_count;
}
```




Problem List
Submit
00:00:00
Premium
Editorial
Solutions
Submissions
Status
Language
Runtime
Memory
Accepted
Sep 29, 2026
0 ms
Testcase
Code
Auto
int findMaxConsecutiveOnes(int* nums, int numsSize
int count, max_count, i;
count = 0;
max_count = 0;
for(i=0; i<numsSize; i++){
if(nums[i]==1)
count += 1;
else{
if(count>max_count)
max_count = count;
count = 0;
}
}
if(count>max_count)
return count;
return max_count;
}
bhanukiranp
Access all features with our Premium subscription!
My Lists
Notebook
Progress
Points
Try New Features
Orders
My Playgrounds
Settings
Appearance
Sign Out
Saved
Ln 1, Col 1
Test Result

Description – Contains Duplicates

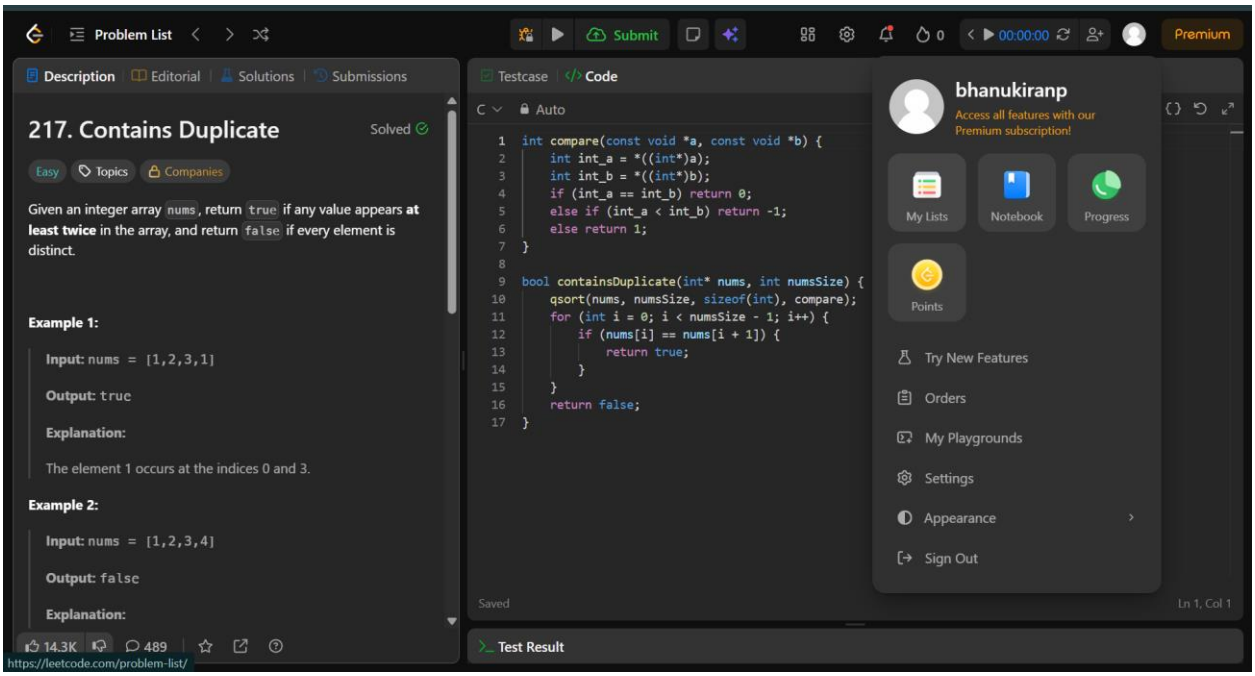
Given an integer array nums, return true if any value appears **at least twice** in the array, and return false if every element is distinct.

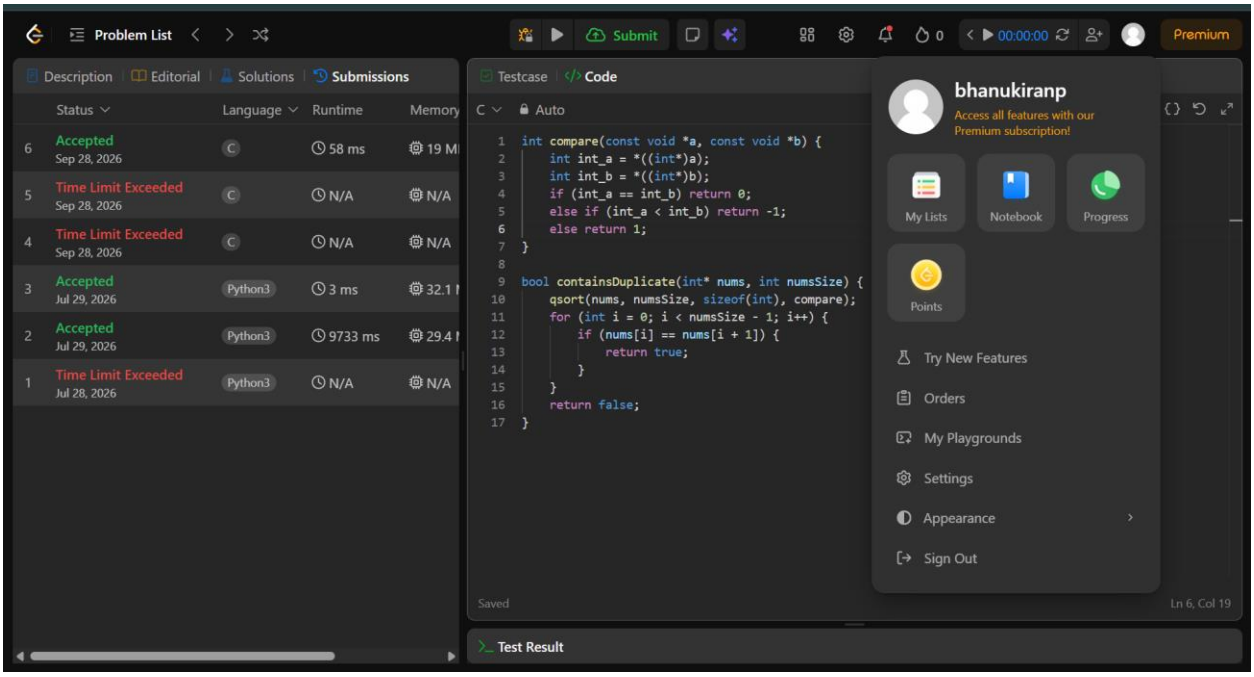
Example 1:
Input: nums = [1,2,3,1]
Output: true
Explanation:
The element 1 occurs at the indices 0 and 3.
Example 2:
Input: nums = [1,2,3,4]
Output: false
Explanation:
All elements are distinct.
Example 3:
Input: nums = [1,1,1,3,3,4,3,2,4,2]
Output: true

Code:

```c
int compare(const void *a, const void *b) {
    int int_a = *((int*)a);
    int int_b = *((int*)b);
    if (int_a == int_b) return 0;
    else if (int_a < int_b) return -1;
    else return 1;
}

bool containsDuplicate(int* nums, int numsSize) {
    qsort(nums, numsSize, sizeof(int), compare);
    for (int i = 0; i < numsSize - 1; i++) {
        if (nums[i] == nums[i + 1]) {
            return true;
        }
    }
    return false;
}
```


Problem List
Submit
Premium
Description
Editorial
Solutions
Submissions
Testcase
Code
C
Auto
217. Contains Duplicate
Solved
Easy
Topics
Companies
Given an integer array nums, return true if any value appears at least twice in the array, and return false if every element is distinct.
Example 1:
Input: nums = [1,2,3,1]
Output: true
Explanation:
The element 1 occurs at the indices 0 and 3.
Example 2:
Input: nums = [1,2,3,4]
Output: false
Explanation:
14.3K
489
https://leetcode.com/problem-list/
int compare(const void *a, const void *b) {
int int_a = *((int*)a);
int int_b = *((int*)b);
if (int_a == int_b) return 0;
else if (int_a < int_b) return -1;
else return 1;
}
bool containsDuplicate(int* nums, int numsSize) {
qsort(nums, numsSize, sizeof(int), compare);
for (int i = 0; i < numsSize - 1; i++) {
if (nums[i] == nums[i + 1]) {
return true;
}
}
return false;
}
bhanukiranp
Access all features with our Premium subscription!
My Lists
Notebook
Progress
Points
Try New Features
Orders
My Playgrounds
Settings
Appearance
Sign Out
Saved
Ln 1, Col 1
Test Result


Problem List
Submit
Premium
Description
Editorial
Solutions
Submissions
Testcase
Code
C
Auto
Status
Language
Runtime
Memory
6 Accepted Sep 28, 2026 C 58 ms 19 M
5 Time Limit Exceeded Sep 28, 2026 C N/A N/A
4 Time Limit Exceeded Sep 28, 2026 C N/A N/A
3 Accepted Jul 29, 2026 Python3 3 ms 32.1
2 Accepted Jul 29, 2026 Python3 9733 ms 29.4
1 Time Limit Exceeded Jul 28, 2026 Python3 N/A N/A
int compare(const void *a, const void *b) {
int int_a = *((int*)a);
int int_b = *((int*)b);
if (int_a == int_b) return 0;
else if (int_a < int_b) return -1;
else return 1;
}
bool containsDuplicate(int* nums, int numsSize) {
qsort(nums, numsSize, sizeof(int), compare);
for (int i = 0; i < numsSize - 1; i++) {
if (nums[i] == nums[i + 1]) {
return true;
}
}
return false;
}
bhanukiranp
Access all features with our Premium subscription!
My Lists
Notebook
Progress
Points
Try New Features
Orders
My Playgrounds
Settings
Appearance
Sign Out
Saved
Ln 6, Col 19
Test Result

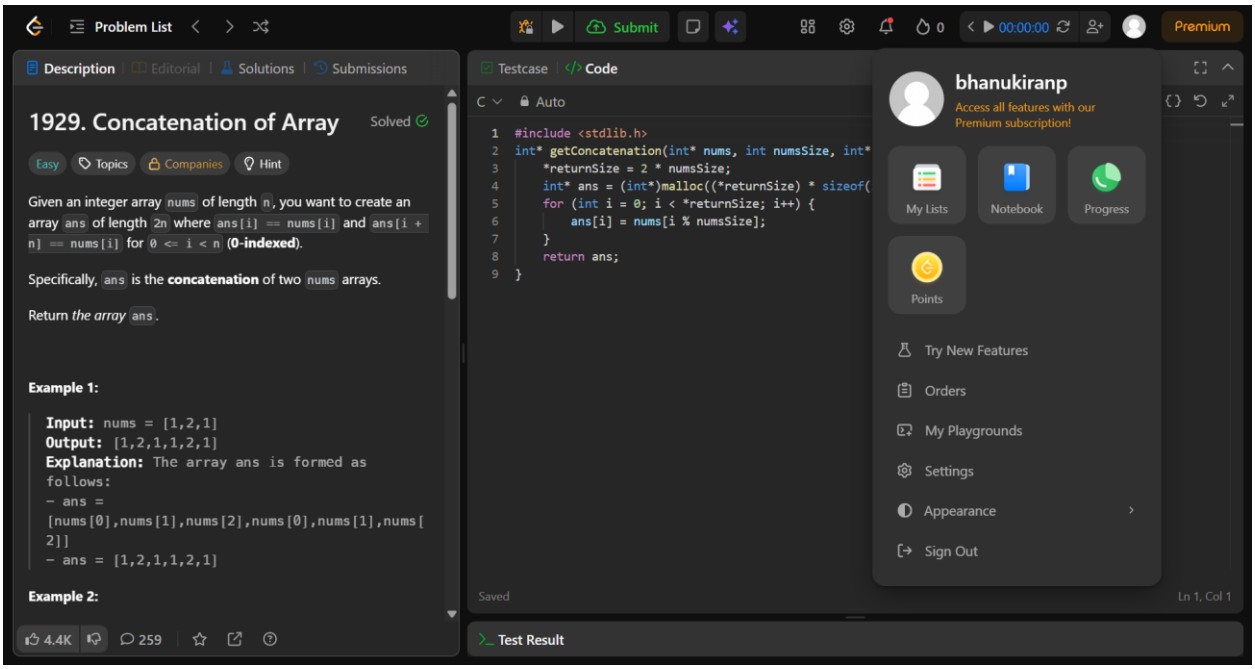
Description – Concatenation Of Array

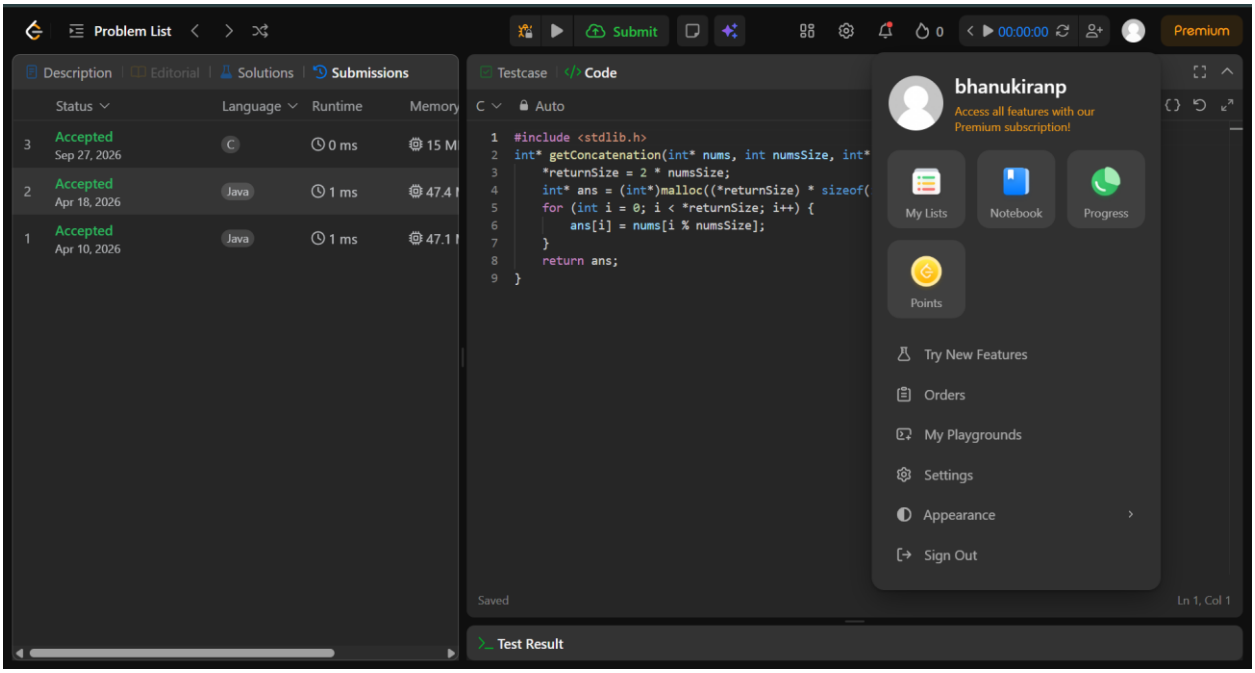
Given an integer array nums of length n, you want to create an array ans of length 2n where ans[i] == nums[i] and ans[i + n] == nums[i] for 0 <= i < n (**0-indexed**).
Specifically, ans is the **concatenation** of two nums arrays.
Return *the array* ans.

Example 1:
Input: nums = [1,2,1]
Output: [1,2,1,1,2,1]
Explanation: The array ans is formed as follows:
- ans = [nums[0],nums[1],nums[2],nums[0],nums[1],nums[2]]
- ans = [1,2,1,1,2,1]
Example 2:
Input: nums = [1,3,2,1]
Output: [1,3,2,1,1,3,2,1]
Explanation: The array ans is formed as follows:
- ans = [nums[0],nums[1],nums[2],nums[3],nums[0],nums[1],nums[2],nums[3]]
- ans = [1,3,2,1,1,3,2,1]

Code:

```c
#include <stdlib.h>
int* getConcatenation(int* nums, int numsSize, int* returnSize) {
    *returnSize = 2 * numsSize;
    int* ans = (int*)malloc((*returnSize) * sizeof(int));
    for (int i = 0; i < *returnSize; i++) {
        ans[i] = nums[i % numsSize];
    }
    return ans;
}
```




Problem List
Submit
00:00:00
Premium
Description
Editorial
Solutions
Submissions
Status
Language
Runtime
Memory
3 Accepted Sep 27, 2026 C 0 ms 15 M
2 Accepted Apr 18, 2026 Java 1 ms 47.4
1 Accepted Apr 10, 2026 Java 1 ms 47.1
Testcase
Code
C
Auto
#include <stdlib.h>
int* getConcatenation(int* nums, int numsSize, int*
*returnSize = 2 * numsSize;
int* ans = (int*)malloc((*returnSize) * sizeof(
for (int i = 0; i < *returnSize; i++) {
ans[i] = nums[i % numsSize];
}
return ans;
}
bhanukiranp
Access all features with our Premium subscription!
My Lists
Notebook
Progress
Points
Try New Features
Orders
My Playgrounds
Settings
Appearance
Sign Out
Saved
Ln 1, Col 1
Test Result

Description- Flipping An Image

Given an n x n binary matrix image, flip the image **horizontally**, then invert it, and return *the resulting image*.
To flip an image horizontally means that each row of the image is reversed.

- For example, flipping [1,1,0] horizontally results in [0,1,1].

To invert an image means that each 0 is replaced by 1, and each 1 is replaced by 0.

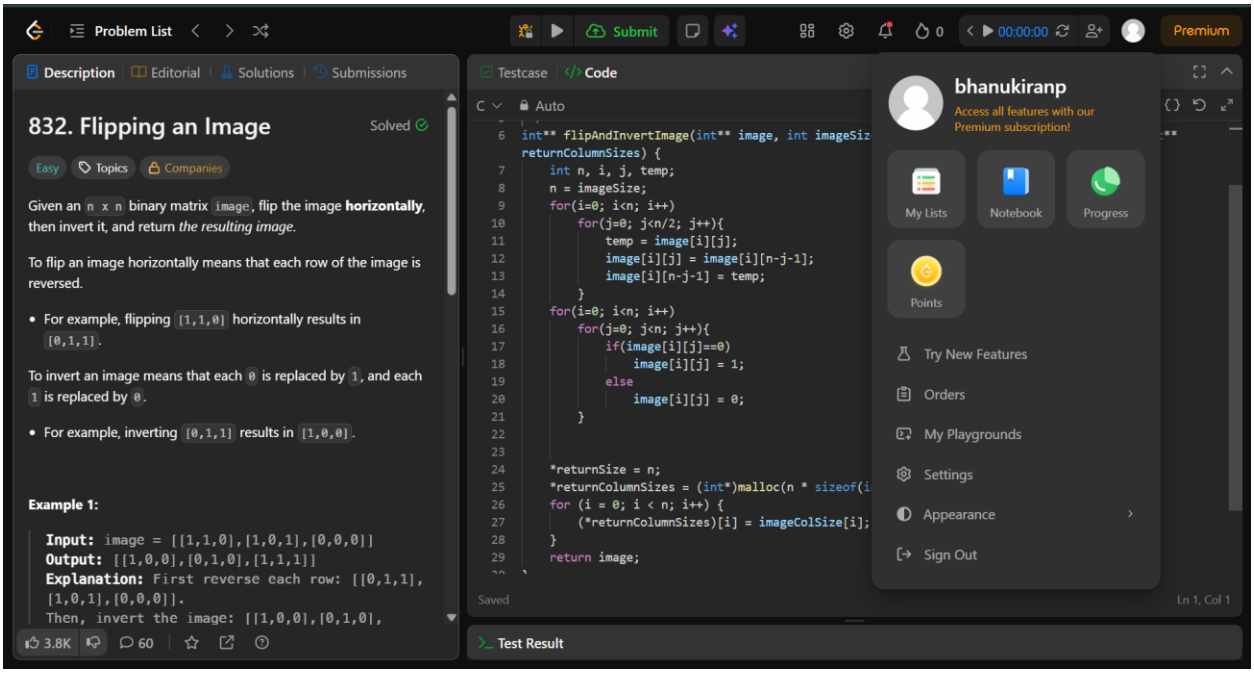
- For example, inverting [0,1,1] results in [1,0,0].

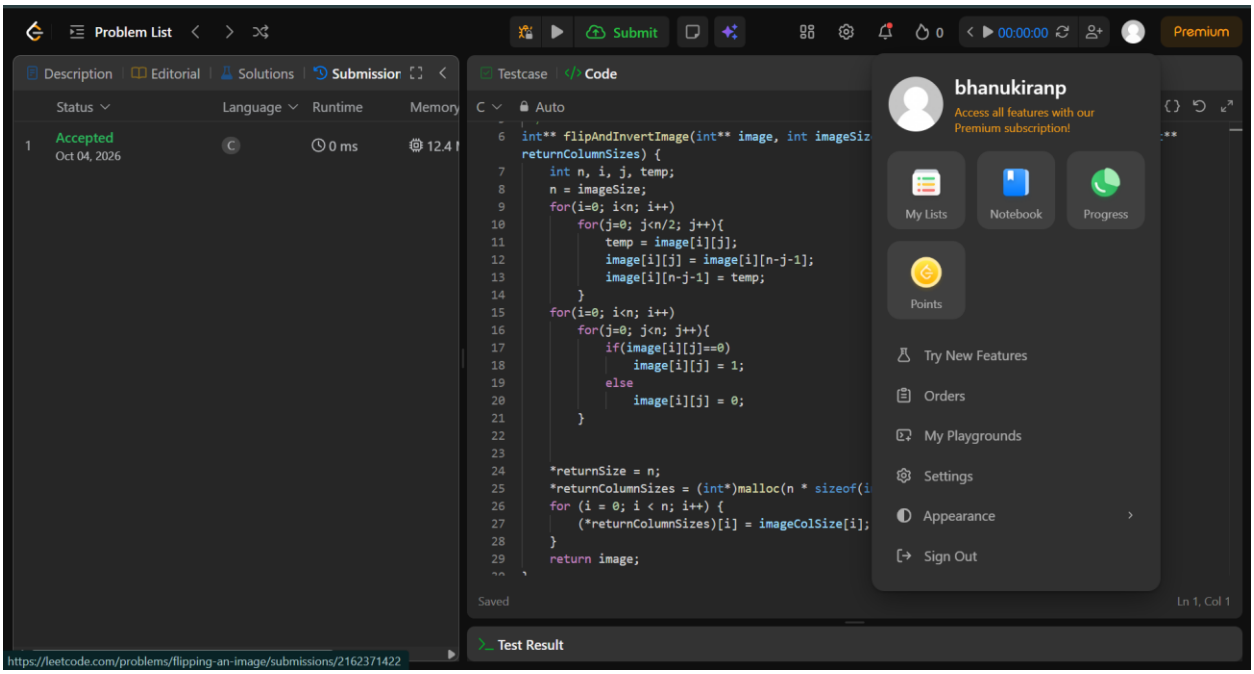
Example 1:
Input: image = [[1,1,0],[1,0,1],[0,0,0]]
Output: [[1,0,0],[0,1,0],[1,1,1]]
Explanation: First reverse each row: [[0,1,1],[1,0,1],[0,0,0]].
Then, invert the image: [[1,0,0],[0,1,0],[1,1,1]]
Example 2:
Input: image = [[1,1,0,0],[1,0,0,1],[0,1,1,1],[1,0,1,0]]
Output: [[1,1,0,0],[0,1,1,0],[0,0,0,1],[1,0,1,0]]
Explanation: First reverse each row: [[0,0,1,1],[1,0,0,1],[1,1,1,0],[0,1,0,1]].
Then invert the image: [[1,1,0,0],[0,1,1,0],[0,0,0,1],[1,0,1,0]]

Code:

```
/**
 * Return an array of arrays of size *returnSize.
 * The sizes of the arrays are returned as *returnColumnSizes array.
 * Note: Both returned array and *columnSizes array must be malloced, assume caller calls free().
 */
int** flipAndInvertImage(int** image, int imageSize, int* imageColSize, int* returnSize, int**
returnColumnSizes) {
    int n, i, j, temp;
    n = imageSize;
    for(i=0; i<n; i++)
        for(j=0; j<n/2; j++){
            temp = image[i][j];
            image[i][j] = image[i][n-j-1];
            image[i][n-j-1] = temp;
        }
    for(i=0; i<n; i++)
        for(j=0; j<n; j++){
            if(image[i][j]==0)
                image[i][j] = 1;
            else
                image[i][j] = 0;
        }


    *returnSize = n;
    *returnColumnSizes = (int*)malloc(n * sizeof(int));
    for (i = 0; i < n; i++) {
        (*returnColumnSizes)[i] = imageColSize[i];
    }
    return image;
}
```


Problem List
Submit
Premium
Description
Editorial
Solutions
Submissions
832. Flipping an Image
Solved
Easy
Topics
Companies
Given an n x n binary matrix image, flip the image horizontally, then invert it, and return the resulting image.
To flip an image horizontally means that each row of the image is reversed.
For example, flipping [1,1,0] horizontally results in [0,1,1].
To invert an image means that each 0 is replaced by 1, and each 1 is replaced by 0.
For example, inverting [0,1,1] results in [1,0,0].
Example 1:
Input: image = [[1,1,0],[1,0,1],[0,0,0]]
Output: [[1,0,0],[0,1,0],[1,1,1]]
Explanation: First reverse each row: [[0,1,1],[1,0,1],[0,0,0]].
Then, invert the image: [[1,0,0],[0,1,0],
3.8K
60
Testcase
Code
Auto
int** flipAndInvertImage(int** image, int imageSiz
returnColumnSizes) {
int n, i, j, temp;
n = imageSize;
for(i=0; i<n; i++)
for(j=0; j<n/2; j++){
temp = image[i][j];
image[i][j] = image[i][n-j-1];
image[i][n-j-1] = temp;
}
for(i=0; i<n; i++)
for(j=0; j<n; j++){
if(image[i][j]==0)
image[i][j] = 1;
else
image[i][j] = 0;
}
*returnSize = n;
returnColumnSizes = (int)malloc(n * sizeof(i
for (i = 0; i < n; i++) {
(*returnColumnSizes)[i] = imageColSize[i];
}
return image;
}
Saved
Ln 1, Col 1
Test Result
bhanukiranp
Access all features with our Premium subscription!
My Lists
Notebook
Progress
Points
Try New Features
Orders
My Playgrounds
Settings
Appearance
Sign Out


Problem List
Submit
Premium
Description
Editorial
Solutions
Submissior
Status
Language
Runtime
Memory
1
Accepted
Oct 04, 2026
0 ms
12.4 M
Testcase
Code
Auto
int** flipAndInvertImage(int** image, int imageSiz
returnColumnSizes) {
int n, i, j, temp;
n = imageSize;
for(i=0; i<n; i++)
for(j=0; j<n/2; j++){
temp = image[i][j];
image[i][j] = image[i][n-j-1];
image[i][n-j-1] = temp;
}
for(i=0; i<n; i++)
for(j=0; j<n; j++){
if(image[i][j]==0)
image[i][j] = 1;
else
image[i][j] = 0;
}
*returnSize = n;
returnColumnSizes = (int)malloc(n * sizeof(i
for (i = 0; i < n; i++) {
(*returnColumnSizes)[i] = imageColSize[i];
}
return image;
}
Saved
Ln 1, Col 1
Test Result
https://leetcode.com/problems/flipping-an-image/submissions/2162371422
bhanukiranp
Access all features with our Premium subscription!
My Lists
Notebook
Progress
Points
Try New Features
Orders
My Playgrounds
Settings
Appearance
Sign Out

Description – Maximum Number of Words in Sentence

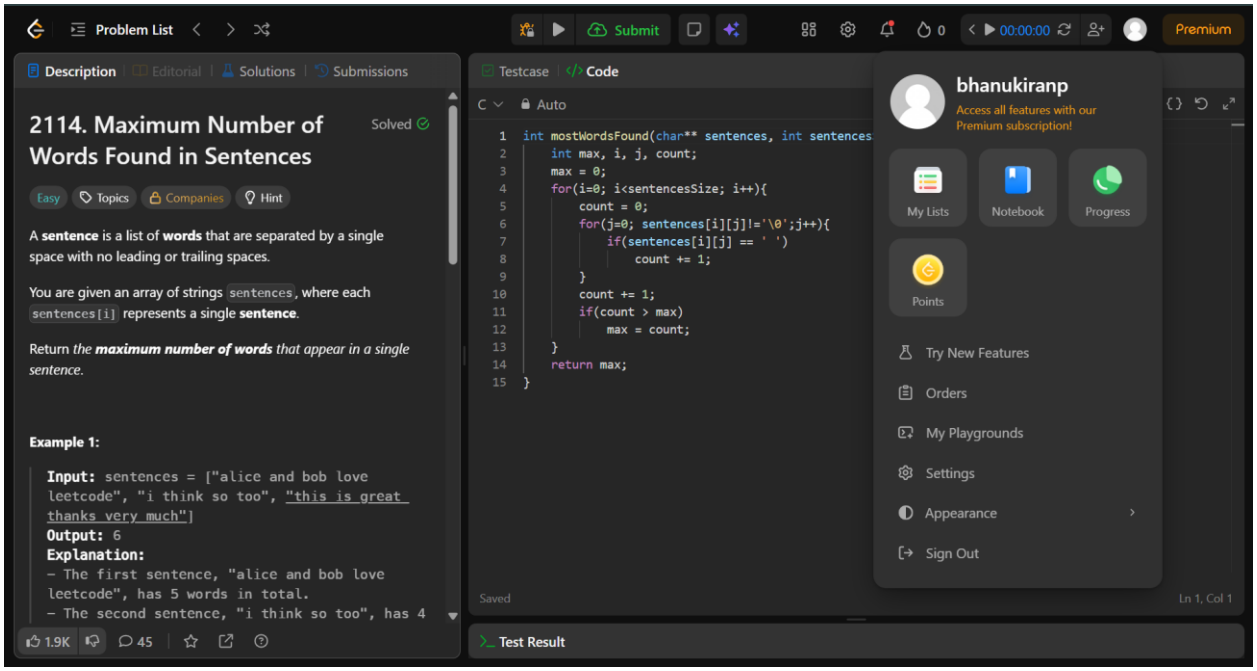
A **sentence** is a list of **words** that are separated by a single space with no leading or trailing spaces.
You are given an array of strings sentences, where each sentences[i] represents a single **sentence**.
Return *the **maximum number of words** that appear in a single sentence*.

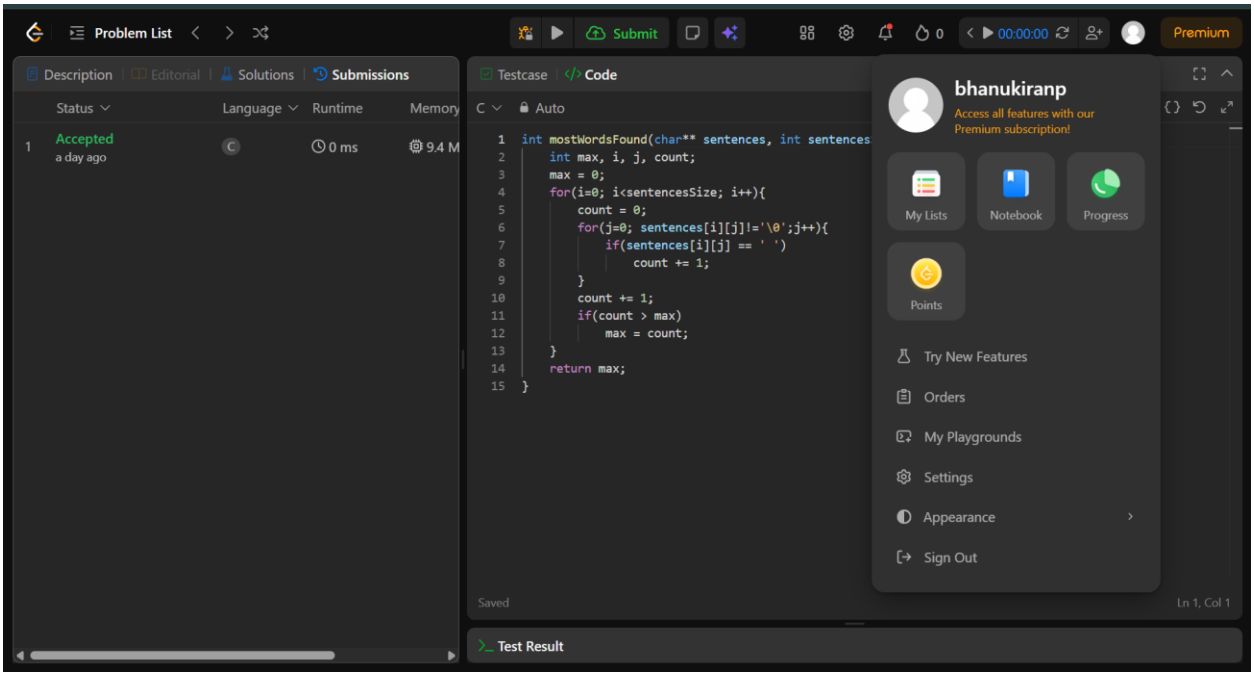
Example 1:
Input: sentences = ["alice and bob love leetcode", "i think so too", "this is great thanks very much"]
Output: 6
Explanation:
- The first sentence, "alice and bob love leetcode", has 5 words in total.
- The second sentence, "i think so too", has 4 words in total.
- The third sentence, "this is great thanks very much", has 6 words in total.
Thus, the maximum number of words in a single sentence comes from the third sentence, which has 6 words.
Example 2:
Input: sentences = ["please wait", "continue to fight", "continue to win"]
Output: 3
Explanation: It is possible that multiple sentences contain the same number of words.
In this example, the second and third sentences (underlined) have the same number of words.

Code:

```
int mostWordsFound(char** sentences, int sentencesSize) {
    int max, i, j, count;
    max = 0;
    for(i=0; i<sentencesSize; i++){
        count = 0;
        for(j=0; sentences[i][j]!='\0';j++){
            if(sentences[i][j] == ' ')
                count += 1;
        }
        count += 1;
        if(count > max)
            max = count;
    }
    return max;
```

```
}
```


Problem List
Submit
Premium
Description
Editorial
Solutions
Submissions
2114. Maximum Number of Words Found in Sentences
Solved
Easy
Topics
Companies
Hint
A sentence is a list of words that are separated by a single space with no leading or trailing spaces.
You are given an array of strings sentences, where each sentences[i] represents a single sentence.
Return the maximum number of words that appear in a single sentence.
Example 1:
Input: sentences = ["alice and bob love leetcode", "i think so too", "this is great thanks very much"]
Output: 6
Explanation:
- The first sentence, "alice and bob love leetcode", has 5 words in total.
- The second sentence, "i think so too", has 4
1.9K
45
Testcase
Code
C
Auto
int mostWordsFound(char** sentences, int sentences
int max, i, j, count;
max = 0;
for(i=0; i<sentencesSize; i++){
count = 0;
for(j=0; sentences[i][j]!='\0';j++){
if(sentences[i][j] == ' ')
count += 1;
}
count += 1;
if(count > max)
max = count;
}
return max;
}
Saved
Ln 1, Col 1
Test Result
bhanukiranp
Access all features with our Premium subscription!
My Lists
Notebook
Progress
Points
Try New Features
Orders
My Playgrounds
Settings
Appearance
Sign Out


Problem List
Submit
Premium
Description
Editorial
Solutions
Submissions
Status
Language
Runtime
Memory
1
Accepted
a day ago
C
0 ms
9.4 M
Testcase
Code
C
Auto
int mostWordsFound(char** sentences, int sentences
int max, i, j, count;
max = 0;
for(i=0; i<sentencesSize; i++){
count = 0;
for(j=0; sentences[i][j]!='\0';j++){
if(sentences[i][j] == ' ')
count += 1;
}
count += 1;
if(count > max)
max = count;
}
return max;
}
Saved
Ln 1, Col 1
Test Result
bhanukiranp
Access all features with our Premium subscription!
My Lists
Notebook
Progress
Points
Try New Features
Orders
My Playgrounds
Settings
Appearance
Sign Out

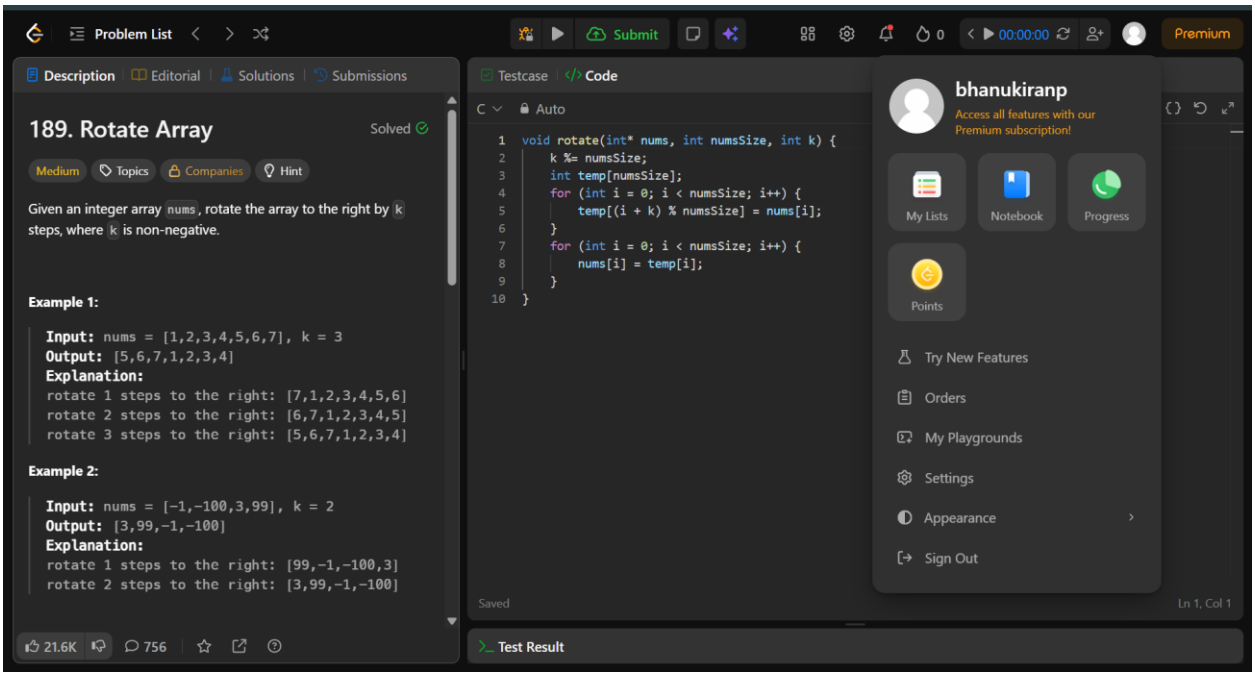
Description – Rotate Array

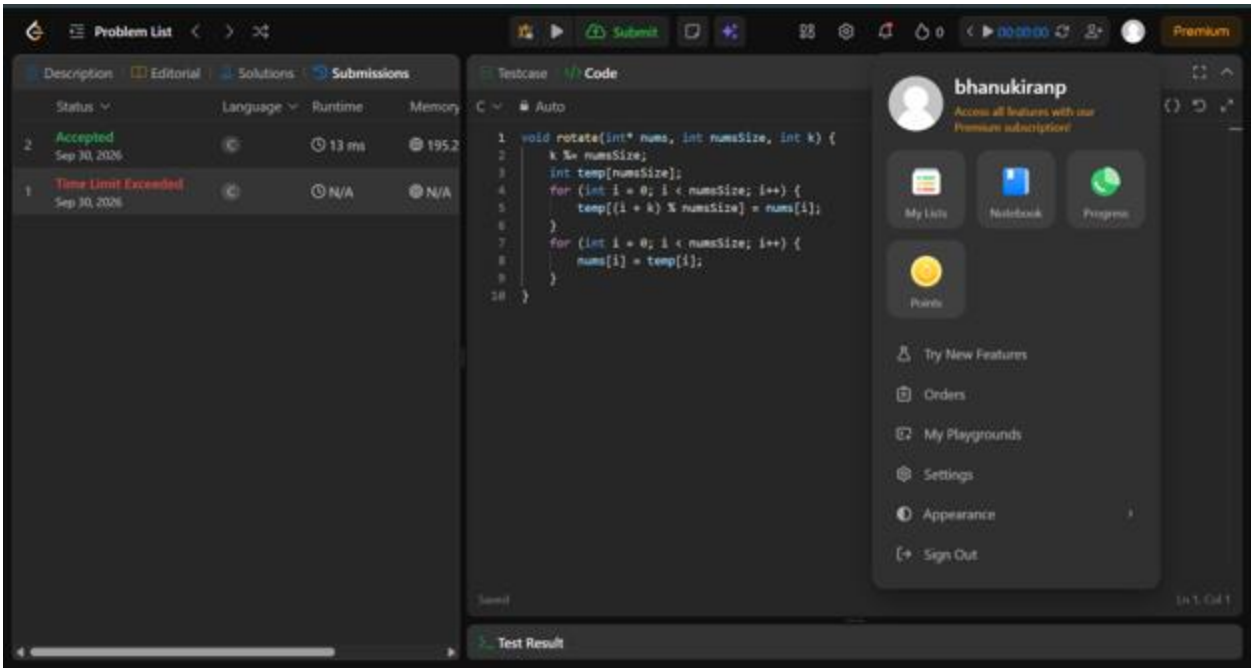
Given an integer array nums, rotate the array to the right by k steps, where k is non-negative.

Example 1:
Input: nums = [1,2,3,4,5,6,7], k = 3
Output: [5,6,7,1,2,3,4]
Explanation:
rotate 1 steps to the right: [7,1,2,3,4,5,6]
rotate 2 steps to the right: [6,7,1,2,3,4,5]
rotate 3 steps to the right: [5,6,7,1,2,3,4]
Example 2:
Input: nums = [-1,-100,3,99], k = 2
Output: [3,99,-1,-100]
Explanation:
rotate 1 steps to the right: [99,-1,-100,3]
rotate 2 steps to the right: [3,99,-1,-100]

Code:

```c
void rotate(int* nums, int numsSize, int k) {
    k %= numsSize;
    int temp[numsSize];
    for (int i = 0; i < numsSize; i++) {
        temp[(i + k) % numsSize] = nums[i];
    }
    for (int i = 0; i < numsSize; i++) {
        nums[i] = temp[i];
    }
}
```




Problem List
Submit
Premium
Description
Editorial
Solutions
Submissions
Testcase
Code
Status
Language
Runtime
Memory
Accepted
Sep 30, 2026
13 ms
195.2
Time Limit Exceeded
Sep 30, 2026
N/A
N/A
C
Auto
void rotate(int* nums, int numsSize, int k) {
k %= numsSize;
int temp[numsSize];
for (int i = 0; i < numsSize; i++) {
temp[(i + k) % numsSize] = nums[i];
}
for (int i = 0; i < numsSize; i++) {
nums[i] = temp[i];
}
}
bhanukiranp
Access all features with our Premium subscription!
My Lists
Notebook
Progress
Points
Try New Features
Orders
My Playgrounds
Settings
Appearance
Sign Out
Test Result

Description – Reverse Words in String

Given an input string s, reverse the order of the **words**.
A **word** is defined as a sequence of non-space characters. The **words** in s will be separated by at least one space.
Return *a string of the words in reverse order concatenated by a single space.*
Note that s may contain leading or trailing spaces or multiple spaces between two words. The returned string should only have a single space separating the words. Do not include any extra spaces.

Example 1:
Input: s = "the sky is blue"
Output: "blue is sky the"
Example 2:
Input: s = " hello world "
Output: "world hello"
Explanation: Your reversed string should not contain leading or trailing spaces.
Example 3:
Input: s = "a good   example"
Output: "example good a"
Explanation: You need to reduce multiple spaces between two words to a single space in the reversed string.

Code:

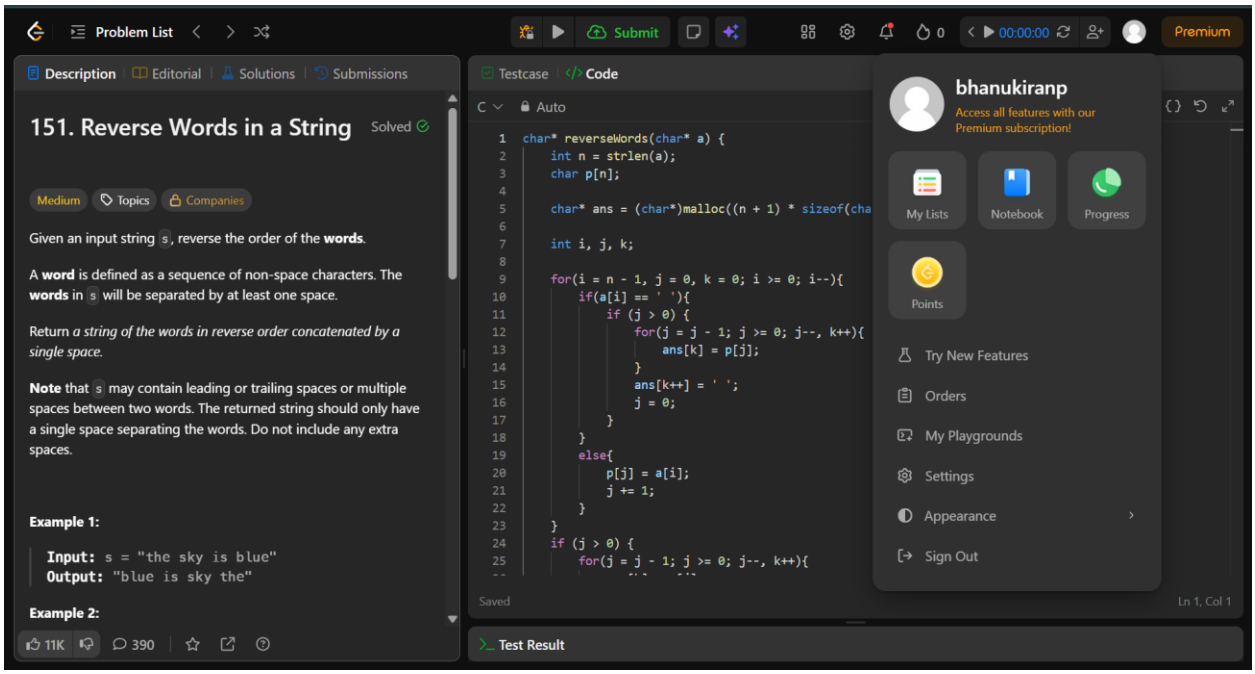
```
char* reverseWords(char* a) {
    int n = strlen(a);
    char p[n];

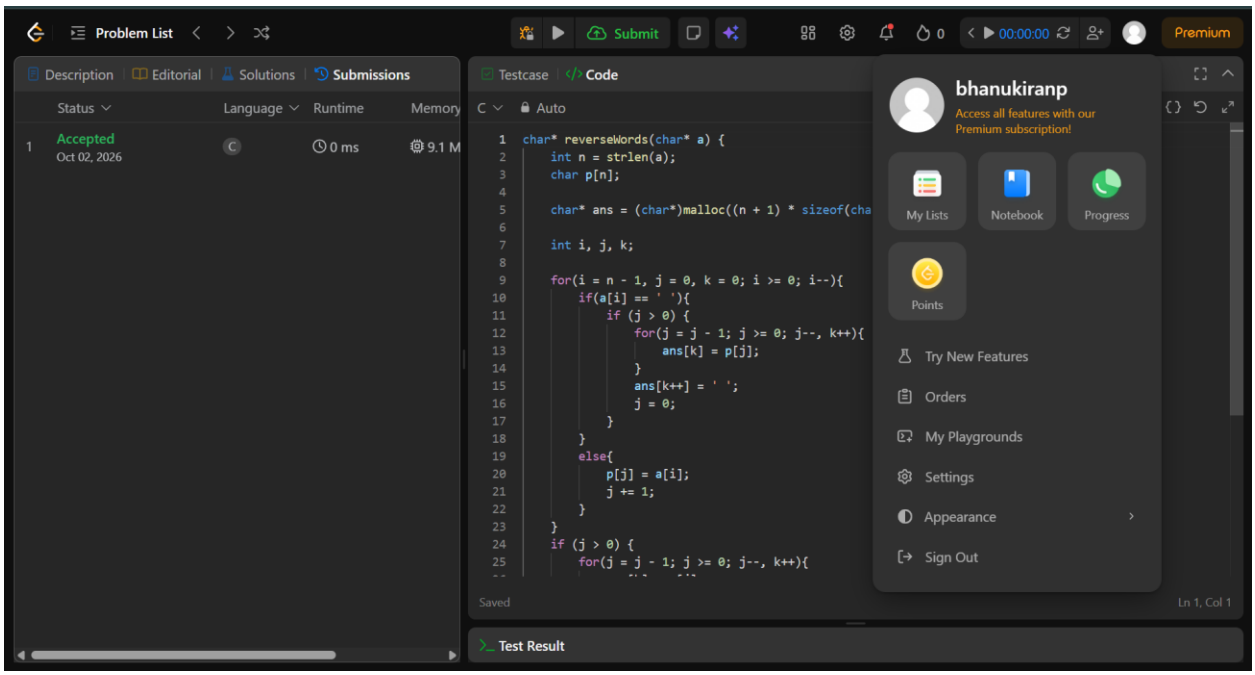
    char* ans = (char*)malloc((n + 1) * sizeof(char));

    int i, j, k;

    for(i = n - 1, j = 0, k = 0; i >= 0; i--){
        if(a[i] == ' '){
            if (j > 0) {
                for(j = j - 1; j >= 0; j--, k++){
                    ans[k] = p[j];
                }
                ans[k++] = ' ';
                j = 0;
            }
        }
        else{
            p[j] = a[i];
            j += 1;
        }
    }
    if (j > 0) {
        for(j = j - 1; j >= 0; j--, k++){
            ans[k] = p[j];
        }
    }
    if (k > 0 && ans[k - 1] == ' ') {
        ans[k - 1] = '\0';
```

```
    } else {
        ans[k] = '\0';
    }
    return ans;
}
```

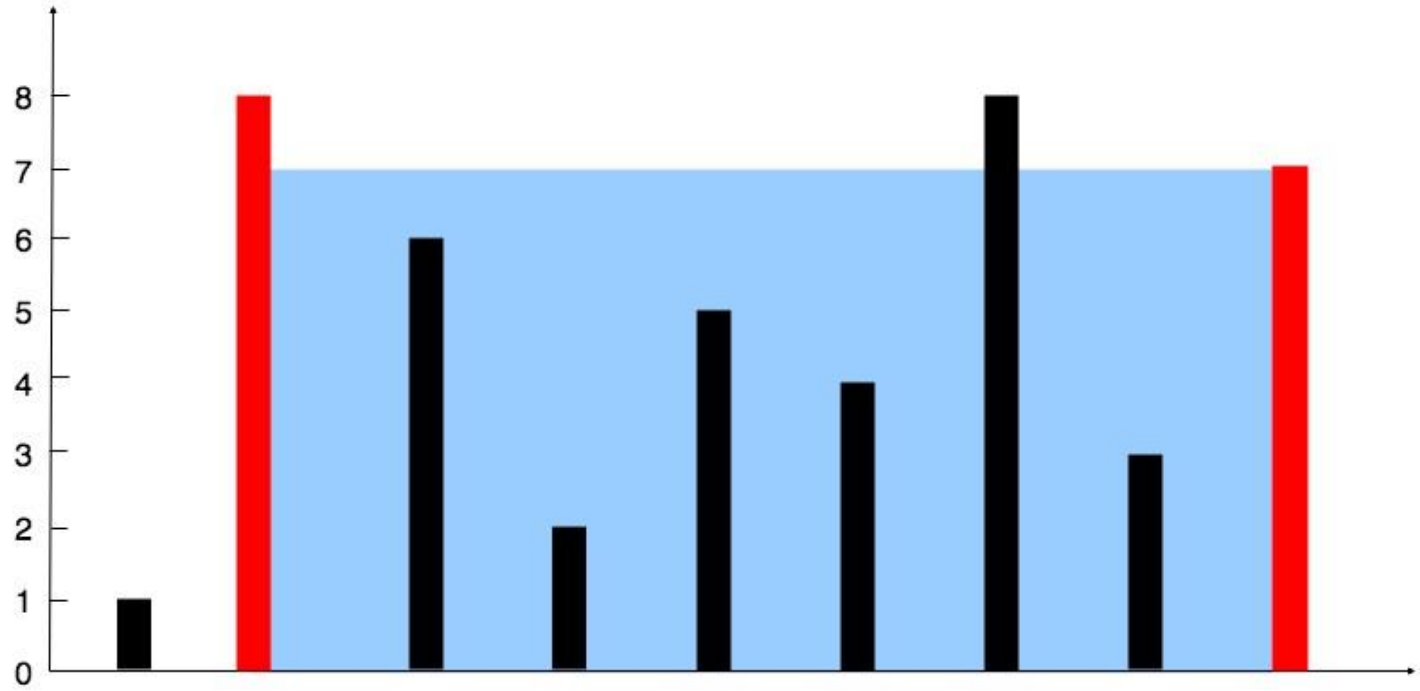




Description – Container With Most Water

You are given an integer array height of length n. There are n vertical lines drawn such that the two endpoints of the ith line are (i, 0) and (i, height[i]).
Find two lines that together with the x-axis form a container, such that the container contains the most water.
Return *the maximum amount of water a container can store*.
Notice that you may not slant the container.

Example 1:

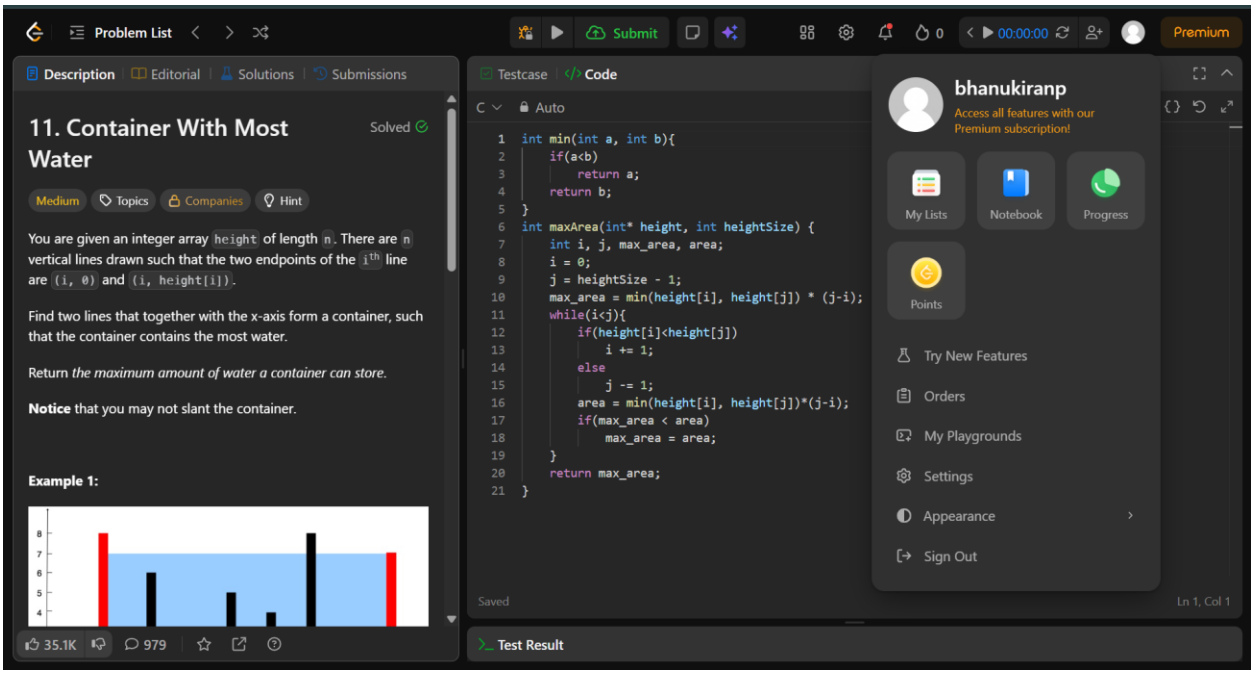


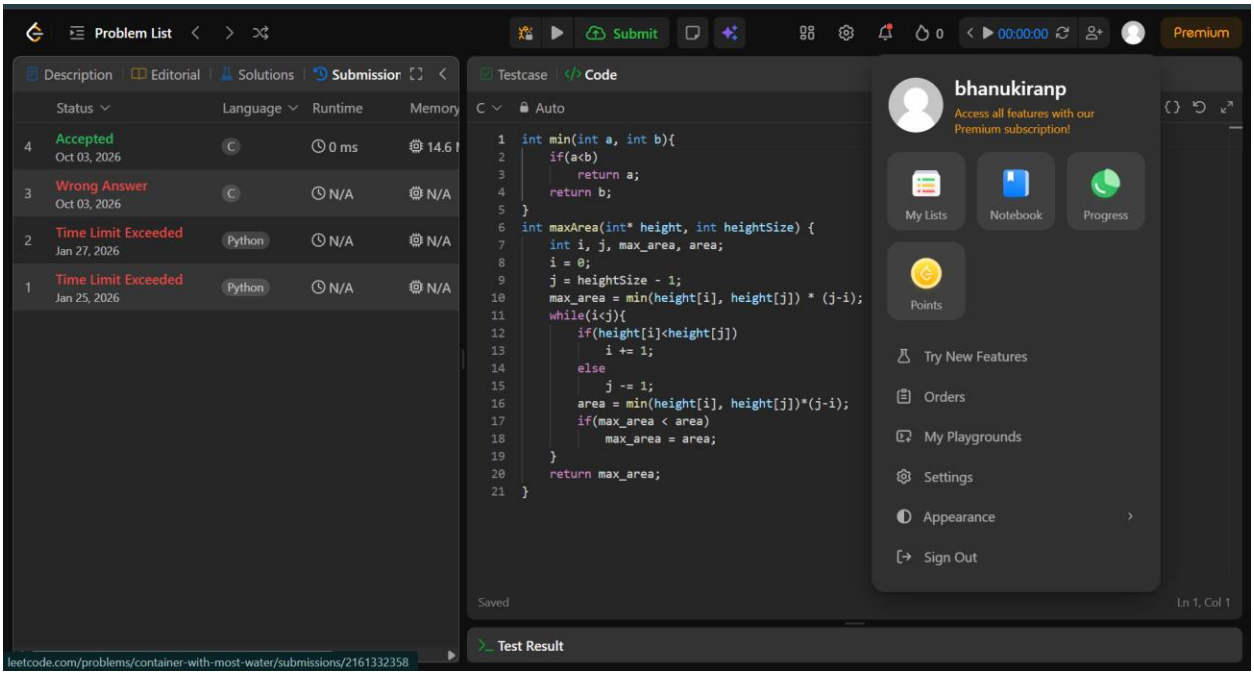
Input: height = [1,8,6,2,5,4,8,3,7]
Output: 49
Explanation: The above vertical lines are represented by array [1,8,6,2,5,4,8,3,7]. In this case, the max area of water (blue section) the container can contain is 49.
Example 2:
Input: height = [1,1]
Output: 1

Code:

```
int min(int a, int b){
    if(a<b)
        return a;
    return b;
}
int maxArea(int* height, int heightSize) {
    int i, j, max_area, area;
    i = 0;
    j = heightSize - 1;
    max_area = min(height[i], height[j]) * (j-i);
    while(i<j){
        if(height[i]<height[j])
            i += 1;
        else
```

```
            j -= 1;
        area = min(height[i], height[j])*(j-i);
        if(max_area < area)
            max_area = area;
    }
    return max_area;
}
```

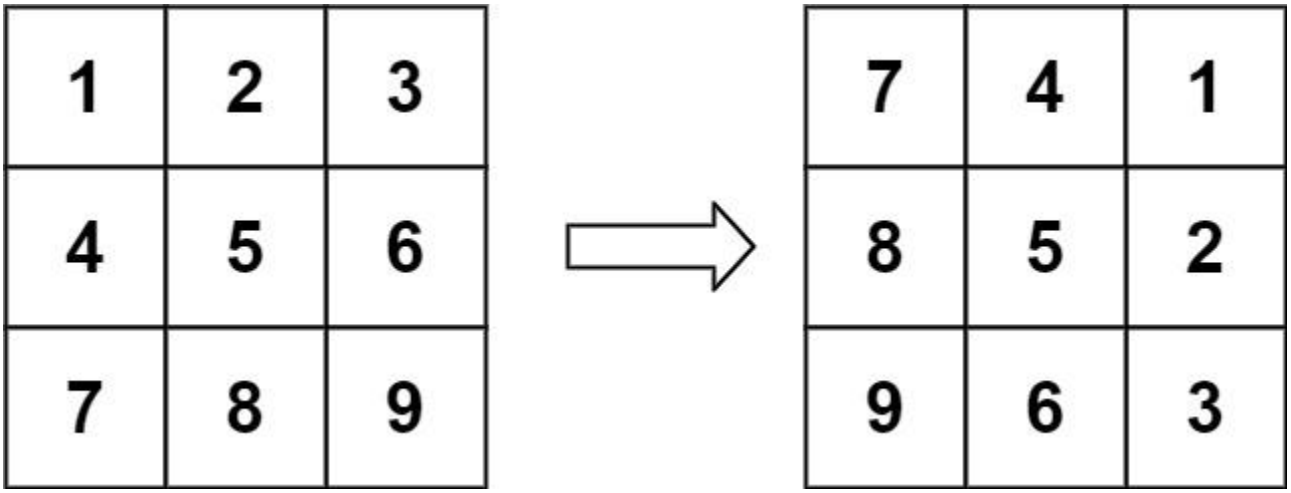




Description – Rotate Image

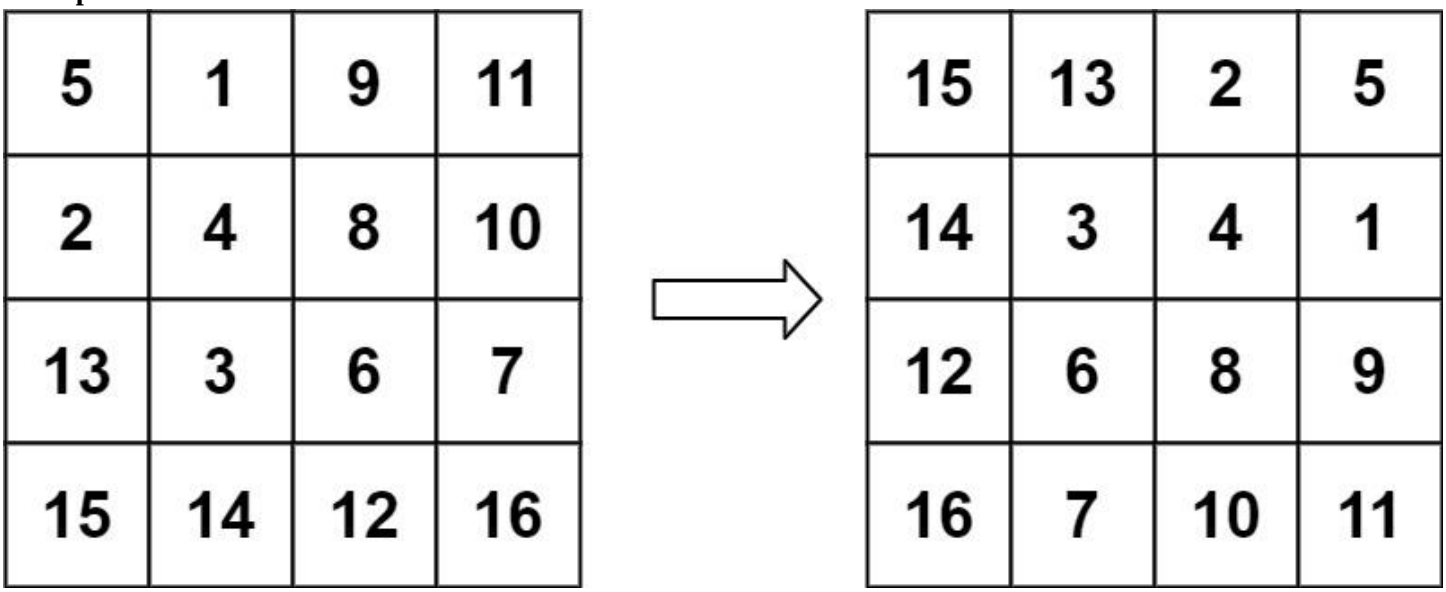
You are given an n x n 2D matrix representing an image, rotate the image by **90** degrees (clockwise).
You have to rotate the image **in-place**, which means you have to modify the input 2D matrix directly. **DO NOT** allocate another 2D matrix and do the rotation.

Example 1:



Input: matrix = [[1,2,3],[4,5,6],[7,8,9]]
Output: [[7,4,1],[8,5,2],[9,6,3]]

Example 2:



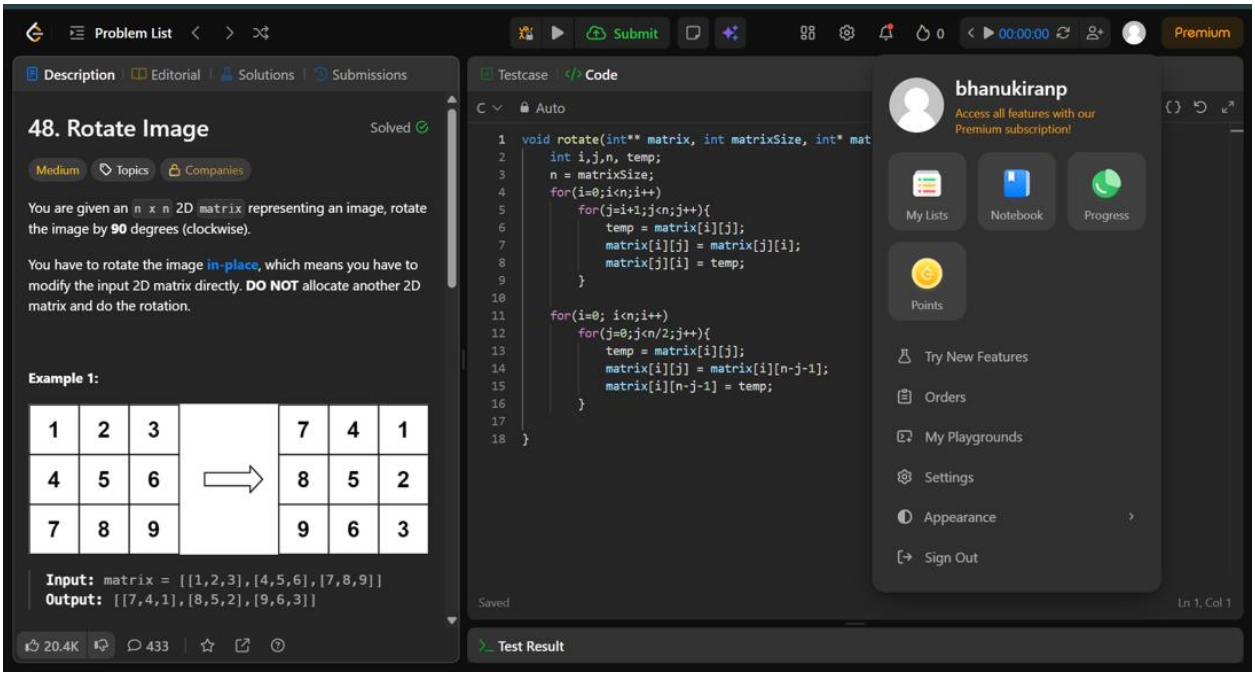
Input: matrix = [[5,1,9,11],[2,4,8,10],[13,3,6,7],[15,14,12,16]]
Output: [[15,13,2,5],[14,3,4,1],[12,6,8,9],[16,7,10,11]]

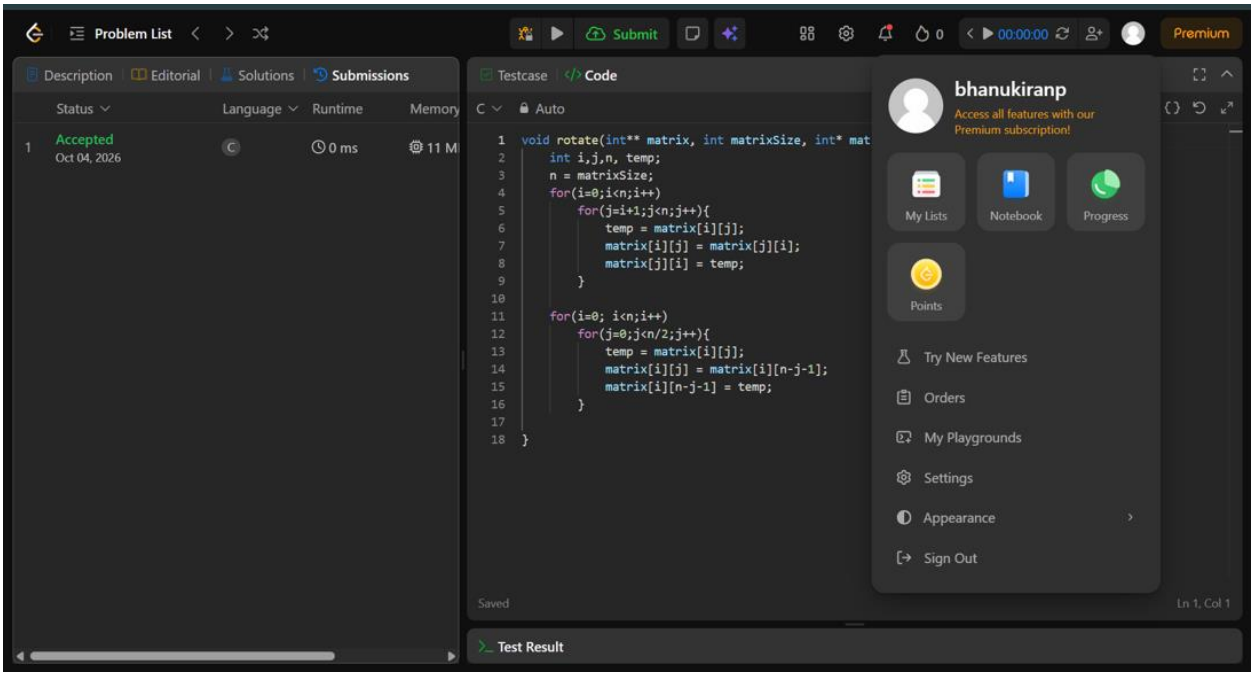
Code:

```
void rotate(int** matrix, int matrixSize, int* matrixColSize) {
    int i,j,n, temp;
    n = matrixSize;
    for(i=0;i<n;i++)
        for(j=i+1;j<n;j++){
            temp = matrix[i][j];
```

```
            matrix[i][j] = matrix[j][i];
            matrix[j][i] = temp;
        }

    for(i=0; i<n;i++)
        for(j=0;j<n/2;j++){
            temp = matrix[i][j];
            matrix[i][j] = matrix[i][n-j-1];
            matrix[i][n-j-1] = temp;
        }
}
```





Description – Longest Substring without Repeating Characters

Given a string s, find the length of the **longest substring** without duplicate characters.

Example 1:
Input: s = "abcabcbb"
Output: 3
Explanation: The answer is "abc", with the length of 3. Note that "bca" and "cab" are also correct answers.
Example 2:
Input: s = "bbbbb"
Output: 1
Explanation: The answer is "b", with the length of 1.
Example 3:
Input: s = "pwwkew"
Output: 3
Explanation: The answer is "wke", with the length of 3.
Notice that the answer must be a substring, "pwke" is a subsequence and not a substring.

Code:

```
int lengthOfLongestSubstring(char* s) {
    int i, j, k, flag, max_count;
    int len = strlen(s);
    char *sub = (char *)malloc((len + 1) * sizeof(char));
    k = 0;
    max_count = 0;
    for(i = 0; s[i] != '\0'; i++) {
        flag = 0;
        for(j = 0; j < k; j++) {
            if(s[i] == sub[j]) {
                flag = 1;
                break;
            }
        }

        if(flag == 0) {
            sub[k++] = s[i];
        } else {
            if(k > max_count) {
                max_count = k;
            }
            int shift = j + 1;
            for (int m = shift; m < k; m++) {
                sub[m - shift] = sub[m];
            }
            k = k - shift;
            sub[k++] = s[i];
        }
    }
    if (k > max_count) {
        max_count = k;
    }
```

```
    free(sub);
    return max_count;
}
```

